

Chart Question Answering with Grounded Facts and Executable Plans

Two-stage LoRA fine-tuning of Qwen3-VL-2B

Ahmed Tamer	8665
Ezz Khaled	8806
Abdelrahman Khalifa	9052

Abstract

A model that answers a chart question with one word gives no way to check the answer. In this project we fine-tune a small vision-language model, Qwen3-VL-2B-Instruct, to answer with a structured record instead. The record contains the chart values the model used, a bounding box for each value, a small program (a *plan*) that computes the answer from those values, and the answer itself. A deterministic executor re-runs the plan, so the computation can be checked.

Training has two stages. Stage 1 teaches grounding with bounding boxes from questions whose evidence is marked. Stage 2 adds plans. For real questions, the plans are written by large language models (LLMs) and kept only if they pass automatic checks, such as reproducing the gold answer.

On a 200-question RefChartQA validation sample, not seen during training and scored with the official evaluator, answer accuracy rises from 57.5% to 87.5% and grounding AP@0.5 from 0.01% to 71.5%. On a 200-question ChartQA validation sample, also unseen during training, accuracy with the structured output rises from 65.0% to 77.0% (81.0% after Stage 1), while plain-answer accuracy stays at 78–79%. After Stage 2, 70.0% of ChartQA questions and 83.5% of RefChartQA questions receive a correct answer that its own plan reproduces. The full validation and test sets were not evaluated because of compute limits.

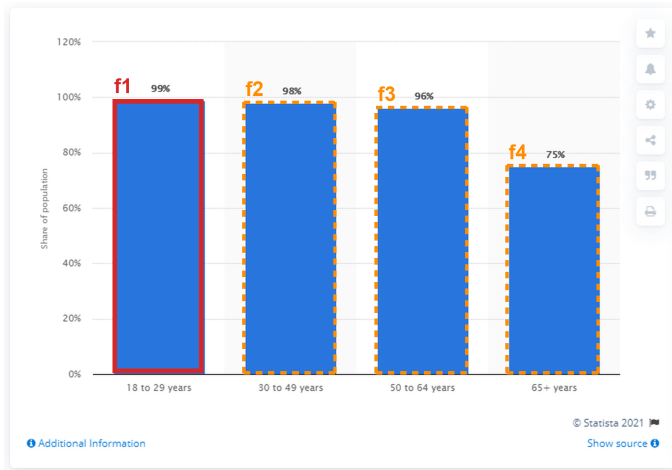
1 Introduction

Chart question answering asks a model to answer a question about a chart image, for example “*What is the difference between 2019 and 2018?*”. Answering takes two steps: reading the right values from the chart, and computing the answer from them. Vision-language models usually return only the final answer. When that answer is wrong, one cannot see whether the model read a wrong value or computed wrongly. When it is right, there is no evidence to confirm it.

This project trains a model to show both steps. For every question the model writes one JSON record (Figure 1) that contains:

- the **facts** it used: each chart value with its label, number, unit, and a bounding box on the chart
- which of those facts answer the question directly (the **evidence**)
- a **plan**: an expression in a small language of 29 operations, which a deterministic executor can run on the facts
- the **answer**

The boxes make the reading step visible and measurable against human annotations. The plan makes the computing step checkable: running it must reproduce the stated answer.



```
{
  "answerable": true,
  "facts":
  [
    {
      "id": "f1",
      "label": "18 to 29 years",
      "value": 99.0,
      "unit": "%",
      "bbox": [160, 206, 294, 786]
    },
    {
      "id": "f2",
      "label": "30 to 49 years",
      "value": 98.0,
      "unit": "%",
      "bbox": [354, 212, 488, 786]
    },
    {
      "id": "f3",
      "label": "50 to 64 years",
      "value": 96.0,
      "unit": "%",
      "bbox": [549, 223, 682, 786]
    },
    {
      "id": "f4",
      "label": "65+ years",
      "value": 75.0,
      "unit": "%",
      "bbox": [742, 346, 876, 786]
    }
  ],
  "evidence_refs": ["f1"],
  "plan":
  {
    "args":
    [
      {"ref": "f1"},
      {"ref": "f2"},
      {"ref": "f3"},
      {"ref": "f4"}
    ],
    "op": "argmax"
  },
  "model_answer": "18 to 29 years"
}
```

Figure 1: Stage 2 output for an actual RefChartQA validation question: “Which age group has highest share of population in 2021?” (gold answer: 18 to 29 years). Left: the boxes of the four facts drawn on the chart. The evidence fact **f1** (solid red) matches the ground-truth box with IoU 0.997, while **f2**, **f3** and **f4** (dashed orange) are context facts that the comparison needs. Right: the output record. The executor runs **argmax** over all four facts and returns “18 to 29 years”, which reproduces the stated answer.

2 Background and related work

ChartQA. ChartQA¹ is a benchmark of real-world charts with two kinds of questions: *human* questions written by people, and *machine* questions generated automatically. Each question has a gold answer. The dataset also provides each chart’s data table and, for bar, line and pie charts, annotations of the chart elements with their boxes. The standard metric is *relaxed accuracy*: a numeric answer is correct if it is within 5% of the gold value. Any other answer must match the gold answer exactly, ignoring case.

RefChartQA. RefChartQA² adds visual grounding: for each question it gives the boxes of the chart regions needed to answer it. Its charts and questions are mainly taken from ChartQA. Its questions come in three subsets: *human* and *machine* questions from ChartQA, and *PoT* questions, which are mostly templated arithmetic questions taken from the TinyChart project. Its official evaluator reports answer accuracy (relaxed), AP@0.5, and P@F1 (defined in Section 6). RefChartQA gives boxes but not what they contain: a box has no label and no value.

Vision-language models and fine-tuning. Qwen3-VL [6] is a family of open vision-language models that can output bounding boxes as coordinates normalised to 0–1000. We fine-tune its 2B-parameter instruction-tuned model with LoRA, which trains a few extra weights and keeps the original weights frozen.

Programs for numerical reasoning. Language models often make mistakes when they do arithmetic and reasoning themselves. A common fix is to let the model write a small program and let a computer run it:

- **PAL** [3] (program-aided language models) solves math word problems. The language model reads the problem and writes a Python program that computes the answer. A Python interpreter runs the program, and its output is the answer. The model does the reasoning, and the interpreter does the arithmetic.
- **Program of Thoughts** [2] uses the same idea for numerical questions about text and tables. The model writes its reasoning steps as code instead of sentences, and the code is run to get the answer.
- **DePlot** [4] works on charts in two steps. First, a model converts the chart image into its data table, written as text. Then a separate large language model reads that table and answers the question, either by reasoning in text or by writing a Python program that is run to get the answer. Reading the chart and reasoning about it are done by two different models.
- **TinyChart** [8] is a small (3B) chart model trained to answer numerical chart questions by writing a Python program, which is then run to get the answer.

Our plans differ in three ways. First, in PAL, Program of Thoughts and TinyChart the numbers are written into the code as constants. A plan’s inputs are instead references to facts, and each fact has a box on the chart, so every number used in the computation can be traced to a place on the chart. Second, a plan is not general Python code. It uses a small language of 29 operations with fixed types, so every plan can be checked automatically. Third, the plan is part of the same

¹Paper: <https://aclanthology.org/2022.findings-acl.177/>. Data: <https://huggingface.co/datasets/ahmed-masry/ChartQA>.

²Paper: <https://arxiv.org/abs/2503.23131>. Data: <https://huggingface.co/datasets/omoured/RefChartQA>.

output as the boxes, so one model reads the chart, points at the evidence and computes the answer in a single output.

Filtering model-written plans by their result. STaR [7] improves a model’s reasoning by letting it write step-by-step explanations for questions, keeping only the explanations that lead to the correct answer, and training the model on the kept ones. Our plan mining follows the same principle, with stricter checks: a teacher’s plan is kept only if executing it reproduces the gold answer and it also passes further checks on its form and operands (Section 4.5).

3 Output format and plan language

3.1 The output record

Every answer is one JSON object with five fields (Figure 1):

- **answerable**: `false` only when the chart does not contain the answer. Then the facts and evidence are empty, the plan is `null`, and the answer is empty.
- **facts**: the chart values the answer uses. Each fact has an id (`f1`, `f2`, ...), the label printed on the chart, a value (number or text), a unit (or `null`), and a box $[x_1, y_1, x_2, y_2]$. Box coordinates are integers from 0 to 999, relative to the image width and height, with (x_1, y_1) the top-left corner.
- **evidence_refs**: the ids of the facts that directly answer the question. Their boxes are the ones scored for grounding. The other facts are *context*: in Figure 1 the plan must compare all four bars, but only the winning bar is evidence.
- **plan**: an expression that computes the answer from the facts.
- **model_answer**: the final answer, as a short string.

The answer is the last field, so the model writes its facts and its plan before it writes the answer.

3.2 The plan language

A plan is a small program that computes the answer from the facts. It is a tree of operations written in JSON. Each operation has a name (`op`) and a list of inputs (`args`). An input is either a reference to a fact, such as `{"ref": "f1"}`, or another operation whose result becomes the input. For example, the question “*What is the difference between the highest and the second highest value?*” on a chart with five bars has the plan

```
difference(max(f1, f2, f3, f4, f5),
  rank_value(f1, f2, f3, f4, f5, position = 2, order = descending))
```

Figure 1 shows how a plan is written in JSON. Table 1 lists all 29 operations, with what each one takes, what it returns and what it computes. The inputs are written as follows:

- f is one fact reference, and F is a list of two or more fact references.
- a and b are numbers: each is a fact reference or an operation that returns a number. A is a list of two or more numbers.
- L is a fact reference or an operation that returns the label of one fact (`label_of`, `argmax`, `argmin`, `rank` or `label_where`).

Table 1: The 29 operations of the plan language, grouped by purpose. The input and setting symbols are explained in Section 3.2.

Operation	Inputs	Settings	Result	What it computes
<i>Read one fact</i>				
lookup	f		value	the value of fact f (a number or text)
label_of	f		label	the label of fact f
<i>Combine two numbers</i>				
difference	a, b		number	$a - b$
absolute_difference	a, b		number	$ a - b $
ratio	a, b		number	a/b
percentage	a, b		number	$100 \cdot a/b$, which is a as a percentage of b
percent_change	a, b		number	$100 \cdot (a - b)/b$, the change from b to a in percent
<i>Aggregate</i>				
sum	A		number	the sum of the numbers
mean	A		number	the average of the numbers
median	F		number	the middle value (the average of the two middle values if the count is even)
min	F		number	the smallest value
max	F		number	the largest value
count	F		number	how many facts are listed
<i>Select</i>				
argmax	F		label	the label of the fact with the largest value
argmin	F		label	the label of the fact with the smallest value
rank	F	k, order	label	the label of the fact at position k when the values are sorted in the given order
rank_value	F	k, order	number	the value at position k when the values are sorted in the given order
<i>Compare two numbers</i>				
compare	a, b		word	greater, less or equal, saying how a compares with b
greater_than	a, b		Yes/No	Yes if $a > b$
less_than	a, b		Yes/No	Yes if $a < b$
equal_to	a, b		Yes/No	Yes if $a = b$
<i>Trend</i>				
trend	F		word	the direction of the values in the listed order: increasing (each value is larger than the one before), decreasing (each is smaller), flat (all are equal), or mixed
<i>Filter by a threshold written in the question</i>				
count_where	F	c, t	number	how many facts pass the test
sum_where	F	c, t	number	the sum of the values that pass
mean_where	F	c, t	number	the average of the values that pass
label_where	F	c, t	label	the label of the only fact that passes
meets_threshold	a	c, t	Yes/No	Yes if a passes the test
<i>Labels</i>				
category_of	L		label	the category part of a label. On a chart with several series, a label such as “Men · 2019” names both the series and the category, and the result is “2019”. A label without a series part is returned unchanged
<i>Other</i>				
unanswerable	none		empty	states that the chart does not contain the answer

Only the inputs a , b , A and L can be other operations. All other inputs must be fact references. Some operations also have settings, written next to their inputs:

- k and order: a position (1, 2, ...) and a sort order (ascending or descending).
- c and t : a comparison (greater, greater or equal, less, less or equal, equal) and a threshold. A value passes the test if “value c t ” is true. For example, with $c = \text{greater}$ and $t = 35$, the values above 35 pass.

Four rules apply to every plan:

- **No written values.** Inputs are never numbers or text typed into the plan. Apart from the settings, the only number a plan may contain is the threshold t , and it must be a number written in the question.
- **Depth.** Operations can be nested at most four levels deep. This is enough for chart questions, which rarely need more than two steps.
- **Units.** Values that are combined or compared must have the same unit. For example, a percentage cannot be added to a dollar amount.
- **Fail instead of guess.** A plan fails, and does not execute, when `argmax`, `argmin` or `rank` meets a tie for the selected value, when `label_where` finds zero or several passing facts, when `sum_where` or `mean_where` finds no passing fact, or when it divides by zero.

3.3 The executor and the plan check

The executor evaluates a plan on the record’s own facts and returns a number, a label, or a word (Yes/No, greater/less/equal, or a trend). The *plan check* compares this result with the stated answer. The check shows that the answer follows from the listed facts by the stated computation. It does not show that the facts were read correctly. The boxes and the answer accuracy measure that.

4 Training data

Figure 2 shows how the training data is built and how each training stage uses it.

4.1 Sources

ChartQA supplies real chart questions with gold answers, each chart’s data table, and labelled chart elements with their boxes. We use its bar and pie charts. Line charts are left out because ChartQA’s boxes for a line chart surround the printed value text beside each point, not the point itself. Line charts are still learned from the synthetic charts (Section 4.4).

RefChartQA supplies questions with gold answers and the boxes of the chart regions needed to answer each question.

No chart from the ChartQA validation or test sets is used in training.

4.2 Giving each RefChartQA box a label and a value

A RefChartQA box says *where* the evidence is, but not *what* it is. ChartQA annotates the same images with labelled, valued elements. We therefore match each RefChartQA box to a ChartQA element of the same image. A match is accepted only if its intersection over union (IoU) is at least 0.9 and clearly better than the second-best candidate. If any box of a question cannot be matched, the whole question is left out rather than guessed. With this rule, about 88% of the

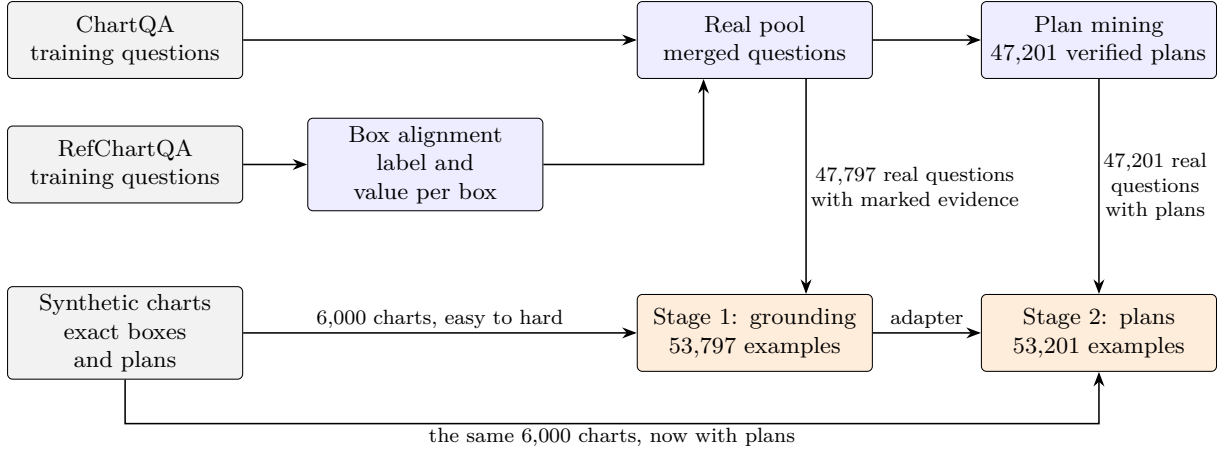


Figure 2: Data and training pipeline. RefChartQA boxes are aligned to ChartQA’s labelled chart elements, and the aligned questions are merged with the ChartQA questions into one real pool. LLM teachers write plans for the pool’s questions, and only verified plans are kept. Each stage trains on real questions plus the same 6,000 synthetic charts: Stage 1 on the 47,797 real questions whose evidence is marked, and Stage 2, which continues from the Stage-1 adapter, on the 47,201 real questions with plans.

RefChartQA questions are aligned. The chart’s other labelled elements are attached as unmarked context facts, which plans can use when a question needs a comparison set (“which is highest?”).

4.3 One real pool

The aligned RefChartQA questions and the ChartQA questions are combined into one pool. Many questions appear in both datasets, because RefChartQA reuses ChartQA’s charts and questions, and each such pair is merged into one record. Each fact takes its value from the chart’s gold data table.

4.4 Synthetic charts

Real questions teach realistic reading. Synthetic charts add complete, exact supervision: because the generator chooses the data, the table, the answer, every box and the plan are known by construction. Each synthetic example is made in five steps:

1. Choose a chart type (vertical bar, horizontal bar, grouped bar, line, multi-line or pie), a difficulty level, and a random style (colours, fonts, size, title, grid, value labels, light or dark background).
2. Choose the data: category labels, such as years or country names, and a value for each. Charts for the harder levels have more data points.
3. Pick a random question template for the chosen level. The template gives the question text and its plan, and the plan is executed on the chosen data to get the answer.
4. Draw the chart with Matplotlib. Matplotlib reports exactly where every drawn element lies in the image, so the box of each element is read directly from Matplotlib.
5. Check every box against the rendered image: it must contain its element and fit it tightly. A chart that fails this check is thrown away.

The questions come at four levels of difficulty:

- **L1:** read one value.

- **L2:** two values: a difference (signed or absolute), a ratio, which one is larger, or a greater/less/equal comparison.
- **L3:** an aggregate over many elements.
- **L4:** a nested plan of two operations.

4.5 Plans from LLM teachers

Real questions have gold answers but no plans. We obtain plans from LLM teachers. For each pool question with complete facts, the teacher sees the question, the gold answer, the chart’s data table, and the ordered list of facts (id, label, value, unit, colour), with the facts marked by RefChartQA tagged. The teacher is given the answer because its task is to *explain* a known answer, not to find it. It replies with exactly one of four things: a plan, a refusal, a request for a missing operation, or a request for missing context. The instructions tell the teacher to choose the operation the question asks for (“which is largest” is **argmax** even if it could name the item directly) and to refuse rather than force a plan.

Every proposed plan must pass six checks, in order, or it is discarded (plans are never repaired):

1. **Shape:** known operations, correct schema, format and operand types.
2. **Stated threshold:** a threshold must be a number written in the question.
3. **Known facts:** every referenced fact exists.
4. **Executes:** the executor runs the plan without error.
5. **Reproduces the gold answer:** executing the plan gives the gold answer.
6. **Covers the marked facts:** where RefChartQA marks the evidence, the plan must use every marked fact. Extra facts are allowed, because an **argmax** needs the whole comparison set even when only the winner is marked.

Mining ran in two rounds. After the first round, the teachers’ requests for missing operations showed common needs, so seven operations were added to the language (**rank_value**, **count_where**, **sum_where**, **mean_where**, **label_where**, **meets_threshold** and **category_of**), and the questions still without a plan were asked again.

In the end, 47,201 questions, 88% of the pool, have a verified plan. DeepSeek V4.1 Flash wrote about two thirds of these plans and GLM-5.3 Flash almost all of the rest. Claude Sonnet 5 and Gemini 3.8 Flash wrote a small number. The most common operation is **lookup**, followed by **sum**, **argmax**, **less_than** and **max**. Most plans are a single operation, and about 5% contain nested operations. The other questions have no plan, for example because their answer is a colour or a series name, which the language cannot express.

5 Model and training

Model. The base model is Qwen3-VL-2B-Instruct. We add LoRA adapters to both parts of the model: the vision encoder, which reads the image, and the language model, which writes the answer. Only the adapters are trained, 24.6 M parameters or about 1% of the model.

Objective. Each example is the prompt with the chart image, followed by the target and the end-of-turn token. The loss is the cross-entropy of the next token, computed only on the target tokens and the end-of-turn token, so the model learns to stop.

Two stages. Stage 1 starts from fresh adapters and learns to ground: to list the right facts with correct boxes, and to answer, with `plan: null`. It trains on the 6,000 synthetic charts, ordered from L1 to L4, followed by the 47,797 real questions whose evidence is marked. This order is a curriculum from easy to hard, so the Stage-1 data is not shuffled. Stage 2 continues training the Stage-1 adapters and learns to write verified plans as well. It trains on the 47,201 real questions with verified plans and the same 6,000 synthetic charts, now with their plans, shuffled together. Stage 2 uses a longer sequence limit because its targets list plan operands and are longer. The prompts are short (Appendix A), because after fine-tuning the format is learned from the data. Table 2 gives the settings.

Checkpoint selection. 200 questions of each stage’s data is kept out of training. Every 1,000 steps and at the end of training, the model answers these 200 held-out questions, and we keep the checkpoint with the highest average of answer accuracy and AP@0.5. In both stages, the best checkpoint was the last one.

6 Evaluation setup

Questions. We use the validation splits only. The test splits were not evaluated because of compute limits. The validation questions were never used for training or for choosing checkpoints, so they still measure performance on unseen questions. On ChartQA we use a fixed sample of 200 validation questions (100 human written, 100 machine generated). On RefChartQA we also use a fixed sample of 200 validation questions. All models answer the same questions.

Models and prompts. We compare three models: the **base** model without adapters, **Stage 1**, and **Stage 2**. For the structured output, the base model gets a longer zero-shot prompt that spells out the format, the list of operations, and three worked examples. Stage 1 and Stage 2 get the prompts they were trained with. To measure plain answering, all three models also get the ChartQA prompt of the Qwen3-VL report, which asks for a single word or phrase.

Reading an output. The record must satisfy the output schema and its rules: all five fields present, unique fact ids, references to existing facts, boxes with $x_1 < x_2$ and $y_1 < y_2$, and a well-formed plan. An output that fails is scored as wrong.

Answer and grounding metrics. ChartQA answers are scored with the official relaxed-accuracy implementation, taken from the Pix2Struct code. RefChartQA outputs are scored with its official evaluator, which receives the boxes of the facts listed in `evidence_refs` and the answer. It reports:

- **accuracy:** relaxed accuracy of the answers.
- **AP@0.5:** average precision at an IoU threshold of 0.5, over the boxes of all questions.
- **P@F1:** the share of questions whose predicted boxes match their ground truth perfectly at IoU 0.5.

Plan metrics. For structured outputs we also count valid outputs, outputs with a plan, plans that *execute* on the output’s own facts, and plans that *reproduce* the stated answer. Two accuracies summarise them, both over all 200 questions:

- **executed-plan accuracy:** the plan’s executed result, taken as the answer, is correct.
- **plan-verified accuracy:** the stated answer is correct *and* its own plan reproduces it.

Table 2: Training settings and runs. Both stages use AdamW without weight decay, a cosine learning-rate schedule, an effective batch of 8 and one epoch.

	Stage 1	Stage 2
Initial adapters	new	Stage 1
LoRA rank, α , dropout	16, 32, 0.05	16, 32, 0.05
Training data (synthetic + real)	6,000 + 47,797	6,000 + 47,201
Data order	easy to hard	shuffled
Prompt	grounding prompt	training prompt
Learning rate	1×10^{-4}	5×10^{-5}
Batch (per step $\times$ accumulation)	4×2	2×4
Sequence limit (tokens)	1,024	1,536
Optimizer steps	6,349	6,587
GPU	H100 NVL 94 GB	H100 SXM 80 GB

7 Results

7.1 Main results

Table 3 gives the main results.

- **Grounding is learned.** The base model’s boxes almost never match the ground truth (AP@0.5 0.01%). After Stage 1, AP@0.5 is 66.65%, and after Stage 2 71.48%. After Stage 2, 75.5% of RefChartQA questions are grounded perfectly (P@F1).
- **RefChartQA answers improve** from 57.5% to 85.5% (Stage 1) and 87.5% (Stage 2).
- **Structured answers reach the accuracy of plain answers.** A structured answer is a much harder task than a plain one. Besides the answer, the model must find every value it uses, draw a correct box around each one, write a plan that computes the answer, and produce valid JSON. An output with any format error counts as wrong. The base model shows how hard this is: asked for the structured record, it scores 14 points lower than with a plain answer (65.0% versus 79.0%), and 26 of its 200 outputs are invalid. After Stage 1 it scores 81.0%, and after Stage 2 77.0%. This is the level of plain answering, but each answer now also comes with its evidence, and after Stage 2 with a plan that can be checked.
- **Plain answering is unchanged** (79.0%, 78.5%, 78.0%): fine-tuning for the structured output did not harm the model’s ordinary answers.

7.2 Results by question kind

Human questions are harder than machine questions for every model (Table 4). On ChartQA, Stage 2 scores 63% on human and 91% on machine questions. On RefChartQA, grounding of human questions (AP@0.5 51.2%) stays well below that of machine and PoT questions (both 80.5%).

7.3 Comparison with published results

Table 5 places our RefChartQA results next to the models compared in the RefChartQA paper. Our grounding is clearly the strongest: Stage 2 has the highest AP@0.5 and P@F1 on all three subsets, for example an AP@0.5 of 51.2 on human questions against at most 32.8. Its answer accuracy is within 4 points of the best model on human questions, and lower on machine questions (94.0 against at most 96.6) and on PoT questions (86.4 against 95.4 for TinyChart-PoT). Two

Table 3: Main results on the validation samples (% , 200 questions each). “Structured” is the JSON record and “plain” is a single word or phrase.

Model	ChartQA accuracy		RefChartQA		
	structured	plain	accuracy	AP@0.5	P@F1
Base	65.0	79.0	57.5	0.01	0.5
Stage 1	81.0	78.5	85.5	66.65	72.0
Stage 2	77.0	78.0	87.5	71.48	75.5

Table 4: Results by question kind (%). ChartQA: structured accuracy, 100 questions per kind. RefChartQA: accuracy / AP@0.5 / P@F1, 66–67 questions per subset.

Model	ChartQA		RefChartQA								
	human	machine	human			machine			PoT		
Base	54	76	67.2 /	0.0 /	0.0	73.1 /	0.1 /	1.5	31.8 /	0.0 /	0.0
Stage 1	70	92	77.6 /	49.6 /	61.2	92.5 /	74.6 /	82.1	86.4 /	77.4 /	72.7
Stage 2	63	91	82.1 /	51.2 /	64.2	94.0 /	80.5 /	85.1	86.4 /	80.5 /	77.3

differences should be kept in mind: our numbers come from a small validation sample rather than the test split, and our model also writes a plan for every answer, which makes its task harder.

On ChartQA, our base model scores 79.0% on the 200-question sample, matching the 79.1% reported for Qwen3-VL-2B-Instruct on the test split [6], which supports our evaluation setup. Our structured answers stay at this level (77–81%) while adding boxes and plans. Other models of about 3B parameters reach 80–84% on the ChartQA test split with plain answers only, for example ChartGemma (80.2%) [5], TinyChart (83.6%) [8] and Qwen2.5-VL (84.0%) [1].

7.4 Do the plans verify the answers?

Stage 2 writes a plan for every valid output (Table 6). On ChartQA, 188 of its 192 plans execute and 167 of those reproduce the stated answer. On RefChartQA, all 197 execute and 179 reproduce it. The share of questions answered correctly *and* verified by the answer’s own plan rises from 53.0% to 70.0% on ChartQA and from 50.0% to 83.5% on RefChartQA. (The base model also writes plans, because its prompt asks for them.)

7.5 Where Stage 2 fails

Stage 2 answers 46 of the 200 ChartQA questions wrongly:

- 8 outputs are invalid (malformed JSON or a malformed plan).
- 11 have a plan whose result differs from the stated answer.
- 27 have a plan that reproduces the wrong answer.

The last group is the largest. In these cases the computation is consistent, and the error lies in the facts the model read or in the operation it chose. On RefChartQA, the 25 wrong answers split into 3, 10 and 12 in the same three groups.

7.6 Stage 2 versus Stage 1

On ChartQA, Stage 2 scores 4 points below Stage 1 with the structured output (77.0% versus 81.0%). On the same 200 questions, only Stage 1 is right on 13, and only Stage 2 on 5. This gap

Table 5: RefChartQA results next to the models compared in the RefChartQA paper (accuracy / AP@0.5 / P@F1, %). The published models were fine-tuned on RefChartQA and scored on its test split. Their accuracy is the one measured when they output boxes and an answer together, as our model does. Our numbers come from the 200-question validation sample (66–67 questions per subset), so the comparison is only indicative.

Model	Size	Human	Machine	PoT
ChartGemma	3B	71.2 / 20.0 / 39.0	94.7 / 60.6 / 77.9	72.1 / 43.4 / 54.9
TinyChart	3B	76.4 / 27.8 / 49.4	96.6 / 71.3 / 84.3	82.3 / 59.7 / 67.6
TinyChart-PoT	3B	86.0 / 25.5 / 46.7	96.2 / 66.1 / 81.9	95.4 / 58.3 / 66.6
Qwen2.5-VL	3B	84.8 / 32.8 / 50.4	96.6 / 59.3 / 76.1	81.7 / 39.3 / 46.9
Ours, Stage 2	2B	82.1 / 51.2 / 64.2	94.0 / 80.5 / 85.1	86.4 / 80.5 / 77.3

Table 6: Plan statistics of the structured outputs (200 questions each). Executed-plan and plan-verified accuracy are over all 200 questions, and answer accuracy is repeated for comparison.

	ChartQA		RefChartQA	
	Base	Stage 2	Base	Stage 2
Valid outputs	174	192	166	197
Outputs with a plan	171	192	164	197
Plans that execute	171	188	164	197
Plans that reproduce the stated answer	134	167	126	179
Answer accuracy (%)	65.0	77.0	57.5	87.5
Executed-plan accuracy (%)	56.0	71.5	55.5	87.0
Plan-verified accuracy (%)	53.0	70.0	50.0	83.5

is small, and its causes are known:

- **It comes from human-written questions.** On machine questions the two stages are equal (92% and 91%). Of the 13 questions that only Stage 1 answers correctly, 11 are human-written.
- **Its causes can be fixed.** Of these 13 questions, 5 are Stage-2 outputs with a format error, and 4 ask for something the plan language cannot express: three about colours (“What’s the color of graph with 56 as the highest value?”) and one about a missing category. Such questions had no verified plan, so they were not in the Stage-2 training data, while Stage 2 always tries to write a plan. Only 4 are wrong readings or wrong operations. Simply using Stage 1’s answer whenever Stage 2’s output is invalid would raise the accuracy.
- **Stage 2 does more for almost the same accuracy.** Stage 1 only grounds and answers. Stage 2 also writes a plan, and for 70.0% of ChartQA questions its answer is correct and reproduced by that plan. Stage 1’s answers cannot be checked this way.
- **On RefChartQA, Stage 2 is better on every metric.** Answer accuracy rises from 85.5% to 87.5%, AP@0.5 from 66.65% to 71.48%, and P@F1 from 72.0% to 75.5%. Grounding kept improving while the model learned to write plans.

8 Limitations

- **Small evaluation samples.** Each benchmark is evaluated on only 200 validation questions. The test splits and the full validation splits were not evaluated because of their compute cost.

- **The plan checks computation, not reading.** A plan run on misread values reproduces a wrong answer, and this is the largest group of Stage-2 errors.
- **Gaps in the plan language.** Answers that are colours, series names or lists, and questions about missing categories, cannot be expressed. Such questions have no plans in the training data, and Stage 2 loses accuracy on them.
- **No real line charts for grounding.** ChartQA’s line-chart boxes do not surround the data points, so real line charts are left out and line charts are learned only from synthetic data.
- **Teacher plans.** The teachers see the gold answer. The checks guarantee that each kept plan reproduces that answer from the chart’s facts, but a plan can still reach the right value by a route that does not match the question’s meaning.
- **One run per stage.** Each stage was trained once, with one seed.

9 Conclusion

We fine-tuned a small vision-language model, Qwen3-VL-2B, to answer chart questions with a checkable record: grounded facts, an executable plan, and the answer. Two LoRA stages, training about 1% of the model’s weights for about five hours on one GPU, achieved the following:

- **The model learned to ground its answers.** RefChartQA AP@0.5 rose from 0.01% to 71.5%, and 75.5% of the questions are grounded perfectly.
- **Its answers became more accurate.** RefChartQA answer accuracy rose from 57.5% to 87.5%.
- **The structured record no longer costs accuracy.** On ChartQA, the base model loses 14 points when it must produce the record. After training, the record is as accurate as a plain answer (77–81% against 78–79%), and plain answering is unchanged.
- **Most answers became checkable.** For 70.0% of ChartQA questions and 83.5% of RefChartQA questions, the answer is correct and reproduced by its own plan, against 53.0% and 50.0% for the base model.

Adding plans in Stage 2 costs a few points on ChartQA, mostly through format errors and questions the plan language cannot yet express, while every RefChartQA metric improves. The next steps follow directly: extend the plan language to colours, series and lists, let the model answer without a plan when none fits, and evaluate on the full test splits.

The trained weights of both stages are available, together with the code, at <https://github.com/Ahmed-Elghazaly/chartqa-dual-target/tree/main/weights>.

References

- [1] S. Bai et al. Qwen2.5-VL technical report, 2025. URL <https://arxiv.org/abs/2502.13923>.
- [2] W. Chen, X. Ma, X. Wang, and W. W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023. URL <https://arxiv.org/abs/2211.12588>.
- [3] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning (ICML)*, 2023. URL <https://arxiv.org/abs/2211.10435>.

- [4] F. Liu, J. M. Eisenschlos, F. Piccinno, S. Krichene, C. Pang, K. Lee, M. Joshi, W. Chen, N. Collier, and Y. Altun. DePlot: One-shot visual language reasoning by plot-to-table translation. In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023. URL <https://arxiv.org/abs/2212.10505>.
- [5] A. Masry, M. Thakkar, A. Bajaj, A. Kartha, E. Hoque, and S. Joty. ChartGemma: Visual instruction-tuning for chart reasoning in the wild, 2024. URL <https://arxiv.org/abs/2407.04172>.
- [6] Qwen Team. Qwen3-VL technical report, 2025. URL <https://arxiv.org/abs/2511.21631>.
- [7] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2203.14465>.
- [8] L. Zhang, A. Hu, H. Xu, M. Yan, Y. Xu, Q. Jin, J. Zhang, and F. Huang. TinyChart: Efficient chart understanding with visual token merging and program-of-thoughts learning. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. URL <https://arxiv.org/abs/2404.16635>.

A Prompts

This appendix gives the exact prompts. In each prompt, {question} is replaced by the question. The JSON template is shown over several lines here for readability. In the real prompts it is one line.

A.1 Grounding prompt (Stage 1)

Used to train Stage 1 and to evaluate it. The model lists the facts with their boxes and answers, with plan: null.

```
Ground the answer to the question in the chart. Reply with one compact JSON object:
{"answerable":<bool>,"facts":[{"id":"f1","label":<str>,"value":<num|str|null>,"unit":<str|null>,"bbox": [x1,y1,x2,y2]}],"evidence_refs":["f1"],"plan":null,
"model_answer":<str>}
Use only question-specific marked facts. bbox is four integers 0-999. plan must be null.

Question: {question}
```

A.2 Training prompt (Stage 2)

Used to train Stage 2 and to evaluate it. The model lists the facts with their boxes, writes a plan, and answers.

```
Answer the question about the chart. Reply with one compact JSON object:
{"answerable":<bool>,"facts":[{"id":"f1","label":<str>,"value":<num|str|null>,"unit":<str|null>,"bbox": [x1,y1,x2,y2]}],"evidence_refs":["f1"],
"plan":{"op":<str>,"args":[{"ref":"f1"}]},"model_answer":<str>}
bbox is four integers 0-999. Plan operands must be fact refs, never raw values.

Question: {question}
```

A.3 Plain prompt (all models)

Used to measure plain answers for the base model, Stage 1 and Stage 2. It is the ChartQA prompt of the Qwen3-VL report, so plain answers are measured the same way as in that report.

```
{question}
Answer the question using a single word or phrase.
```